Java Backend Architecture

Interview Series

Chapter 15.10
Microservices Anti-patterns & Migration

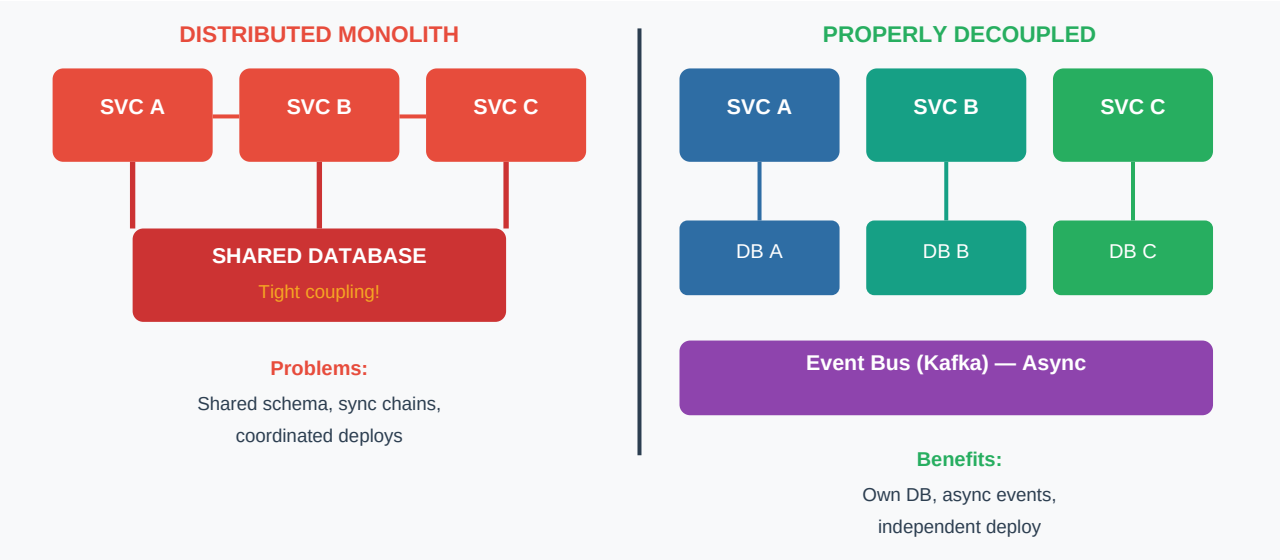
50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

Chapter 15.10 – Microservices Anti-patterns & Migration

Overview

Not all "microservices" are truly decoupled. Common anti-patterns—distributed monolith, chatty services, shared databases—create systems with microservice complexity but monolith rigidity. Recognizing and resolving these anti-patterns, alongside systematic migration strategies like Strangler Fig and dual-write data migration, is a critical senior engineering skill.

Anti-pattern: Distributed Monolith



- Services share a single database — any schema change breaks all services
- Synchronous call chains: Service A → B → C → D creates deep coupling
- Coordinated deployments required — defeats the purpose of microservices
- Symptoms: services that cannot be deployed independently, shared DB tables

Diagnosis

- Check: can each service be deployed without coordinating with other teams?
- Check: does a schema change in one service require changes elsewhere?
- Check: does your service call chain exceed 3 hops synchronously?
- Check: do services import each other's domain objects directly?

Anti-pattern: Chatty Services

- Too many fine-grained synchronous calls between services
- N+1 problem at service level: fetch list, then call service once per item
- High latency, tight availability coupling, network overhead
- Solutions: GraphQL federation (compose in one query), BFF aggregation, batch APIs

```
// Anti-pattern: N+1 service calls List<Order> orders = orderService.getOrders(userId); for
(Order o : orders) { o.setUser(userService.getUser(o.getUserId())); // N calls! } // Better:
```

```
batch call or BFF aggregation Map<String,User> users = userService.getUsers(userIds); // 1 call
// Or: BFF composes order + user in one response
```

Anti-pattern: Shared Database

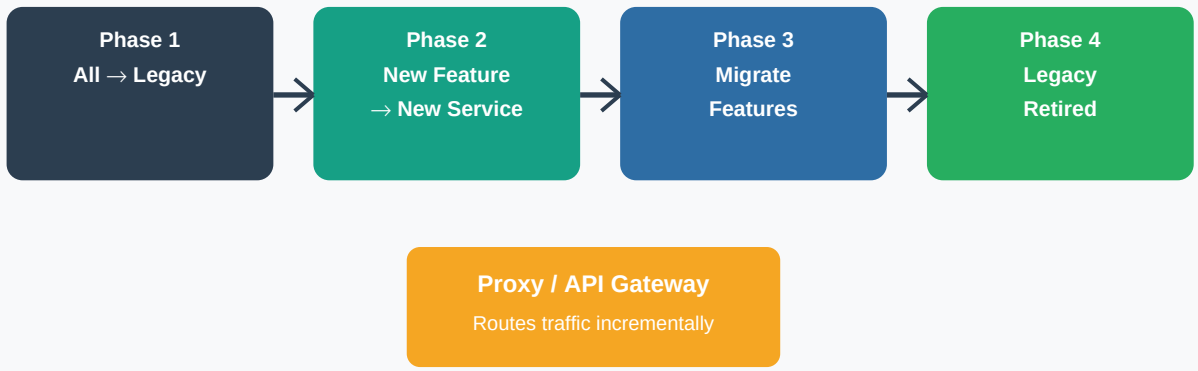
- Multiple services write/read same tables — invisible coupling
- Schema changes must be coordinated across all dependent services
- Breaking the shared DB: assign table ownership, add API facade, migrate consumers
- Each service should own its data; others access via API or events

```
// Shared DB anti-pattern orderService → orders table (writes) reportService → orders table
(reads) // WRONG! // Fix: reportService subscribes to OrderCreated events // and maintains its
own read model (CQRS projection)
```

BFF (Backend for Frontend) Pattern

- Dedicated backend per client type (mobile BFF, web BFF, 3rd-party BFF)
- Aggregates multiple microservice calls into one client-optimized response
- Handles data transformation, filtering, and composition for each client
- Prevents over-fetching/under-fetching in mobile clients

Migration: Strangler Fig Pattern



Strangler Fig: Incremental Migration via Traffic Routing

The Strangler Fig pattern incrementally replaces a legacy monolith by routing feature-by-feature traffic to new microservices. An API gateway or proxy routes traffic, allowing the legacy system to "die" gradually rather than a big-bang rewrite.

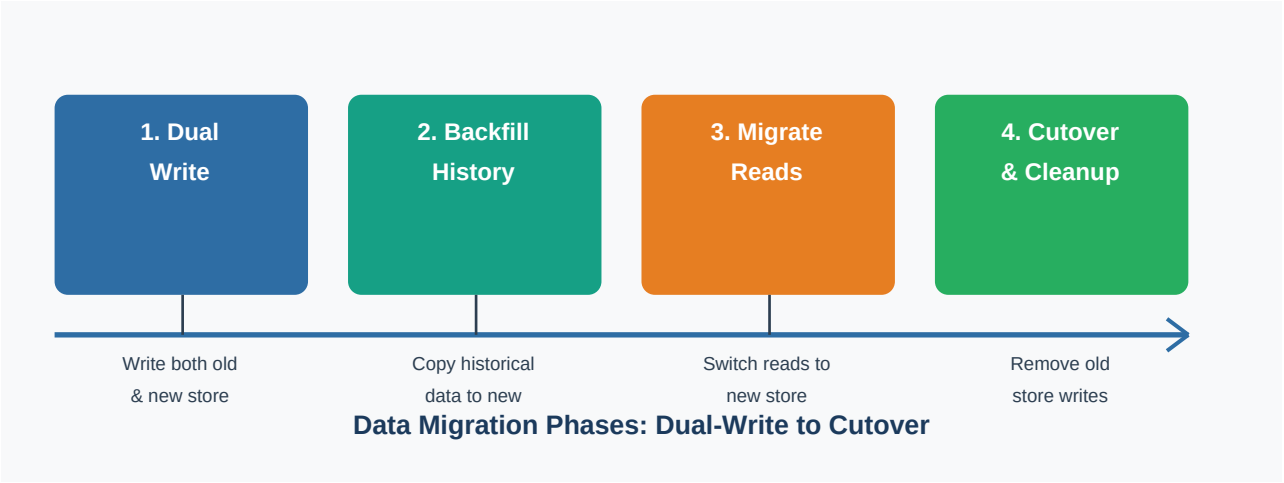
- Step 1: Identify a feature to extract (start with non-critical, well-defined boundaries)
- Step 2: Build new service alongside legacy — do not modify legacy
- Step 3: Route traffic for that feature to new service via proxy/gateway
- Step 4: Monitor, compare outputs, validate parity
- Step 5: Repeat for next feature; gradually legacy handles less traffic
- Step 6: Decommission legacy when all features migrated

Feature Flag Cutover Strategy

- Use feature flags to route X% of traffic to new service (canary)
- Gradually increase percentage (1% → 10% → 50% → 100%)
- Rollback = flip flag back; no redeployment needed
- Parallel run: run both old and new, compare results before cutover

```
@Service public class OrderService { @Autowired FeatureFlagService flags; public Order createOrder(OrderRequest req) { if (flags.isEnabled("use-new-order-service", req.getUserId())) { return newOrderService.create(req); } return legacyOrderService.create(req); } }
```

Data Migration: Dual-Write Pattern



Data Migration Phases: Dual-Write to Cutover

- Phase 1 — Dual Write: write to both old and new store; reads still from old
- Phase 2 — Backfill: copy all historical data to new store
- Phase 3 — Migrate Reads: switch reads to new store, validate correctness
- Phase 4 — Cutover: stop writing to old store; cleanup old schema
- Risk: dual-write introduces consistency window — handle with idempotency and reconciliation

```
@Transactional public void saveOrder(Order order) { // Phase 1 & 2: dual-write legacyOrderRepo.save(order); // old store newOrderRepo.save(toNewModel(order)); // new store // Phase 3: switch reads to newOrderRepo // Phase 4: remove legacyOrderRepo.save() }
```

ArchUnit: Enforcing Module Boundaries

- ArchUnit validates architectural rules as JUnit tests
- Enforce: domain layer must not depend on infrastructure layer
- Enforce: services must not directly reference other services' domain objects
- Prevent accidental tight coupling from being introduced via PRs

```
@AnalyzeClasses(packages = "com.example.orders") class OrderArchitectureTest { @ArchTest static final ArchRule domainNoInfra = noClasses() .that().resideInAPackage("..domain..") .should().dependOnClassesThat() .resideInAPackage("..infrastructure.."); @ArchTest static final ArchRule noUserDomainDep = noClasses() .that().resideInAPackage("com.example.orders..") .should().dependOnClassesThat() .resideInAPackage("com.example.users.domain.."); }
```

Service Mesh Introduction

- Service mesh (Istio, Linkerd) handles cross-cutting concerns at infrastructure level
- Sidecar proxy (Envoy) intercepts all inter-service traffic

- Provides: mTLS, retries, circuit breaking, traffic splitting, distributed tracing
- Enables canary deployments via traffic weighting (90% v1, 10% v2) in Kubernetes
- Reduces application-level complexity — remove Resilience4j if mesh handles it

Anti-patterns Summary Table

Anti-pattern	Symptom	Solution
Distributed Monolith	Shared DB, sync chains, coordinated deploys	Own DB per service, async events
Chatty Services	N+1 HTTP calls, high latency	BFF aggregation, GraphQL, batch APIs
Shared Database	Cross-service table access	Table ownership + API or event
Mega Service	Service too large to deploy fast	Split by bounded context
Nano Services	Too many trivial services	Merge related functionality
Sync Call Chains	A→B→C→D depth	Async events, CQRS, cache

Interview Q&A — Chapter 15.10: Anti-patterns & Migration

Q1. What is a distributed monolith and how do you recognize it?

A distributed monolith looks like microservices but behaves like a monolith. Symptoms: services share a database (or multiple tables in one DB), synchronous call chains span multiple services (A→B→C→D), all services must be deployed together for a release, schema changes require coordinating multiple teams, failure of one service cascades and brings down others.

Q2. What are the root causes of the distributed monolith anti-pattern?

1. Database not split per service—teams took the easy path of sharing existing DB. 2. Business logic leaks across service boundaries (service knows too much about other domains). 3. Synchronous REST calls everywhere instead of async events. 4. Teams aligned by technology (frontend/backend) rather than business capability. 5. Missing bounded context analysis during decomposition.

Q3. How do you fix a distributed monolith?

1. Identify bounded contexts using DDD—assign table ownership per service. 2. Add anti-corruption layers: API facades for cross-service data access. 3. Replace synchronous call chains with async event publishing. 4. Use CQRS+event sourcing where services need cross-domain data (own read projections). 5. Introduce Strangler Fig to decompose incrementally rather than big-bang rewrite. 6. Enforce boundaries with ArchUnit in CI to prevent regression.

Q4. What is the chatty services anti-pattern?

Chatty services make too many fine-grained synchronous calls between services. The N+1 problem at service level: fetch 100 orders, then call user-service 100 times for user details. This causes: high latency (each hop adds network round-trip), tight availability coupling (upstream unavailability cascades), excessive serialization/deserialization overhead. Solutions: batch APIs, BFF aggregation, GraphQL federation, or moving data co-location.

Q5. What is the BFF (Backend for Frontend) pattern?

BFF creates a dedicated backend service per client type (mobile app, web app, third-party API). Each BFF aggregates calls to multiple microservices and returns a client-optimized response. Benefits: mobile BFF returns only fields mobile needs (reduces payload), web BFF can make multiple parallel calls and compose them, teams own their BFF (frontend team owns mobile BFF). Prevents chatty service pattern for client→microservice communication.

Q6. Explain the shared database anti-pattern and its consequences.

Multiple services directly access the same database schema. Consequences: schema evolution requires coordinating all dependent services, one service's data model leaks to others (tight coupling), cannot independently scale or replace a service's storage, transaction management becomes impossible (cross-service 2PC), breaking change in one table breaks all services using it. Fix: each service owns its schema; others access data via API or events.

Q7. How do you break a shared database dependency?

1. Identify which service is the owner of each table (data domain). 2. Add a REST API facade in the owning service for all data access. 3. Migrate non-owning services to use the API instead of direct DB access. 4. For read-heavy consumers: publish events from owner, consumer maintains own read projection (CQRS). 5. Once all consumers migrated, remove cross-service DB access grants. 6. Enforce with network policies (DB only accessible by owning service pod).

Q8. What is the Strangler Fig pattern?

Named after the Strangler Fig tree that wraps and eventually replaces its host. Strategy for migrating legacy systems without big-bang rewrite: 1. Deploy new service alongside legacy. 2. Use a proxy/gateway to route traffic for a specific feature to the new service. 3. Validate new service behavior matches legacy. 4. Incrementally migrate features until legacy handles no traffic. 5. Decommission legacy. The key: never rewrite the legacy; just route traffic away from it.

Q9. What are the steps to implement Strangler Fig in practice?

1. Map legacy features to potential bounded contexts. 2. Select first feature to extract: low risk, clear boundaries, no shared state. 3. Build new service independently; do not modify legacy. 4. Configure gateway routing: route /feature/* to new service, rest to legacy. 5. Run parallel: log both responses, compare for discrepancies. 6. Use feature flag to control what percentage of traffic routes to new service. 7. Gradually increase until 100%. 8. Remove legacy feature.

Q10. What is parallel run (dark launching) in migration?

Parallel run: both the legacy and new implementation process each request simultaneously. Responses are compared; the legacy response is returned to the user. Discrepancies are logged for investigation. This provides confidence before cutover. Also called 'dark launching' (new service runs in shadow, not visible to users). In Java: use DiverterService that calls both, logs differences, returns from primary (legacy).

Q11. How do feature flags support Strangler Fig migration?

Feature flags control traffic routing without code deployment. Canary release: route 1% of traffic to new service initially. Gradually increase: 1% → 10% → 50% → 100% while monitoring error rates and latency. Rollback = change flag value instantly without redeployment. Targeting: route by user ID, region, or account tier for controlled exposure. Tools: LaunchDarkly, Unleash, or Spring Cloud config-based flags.

Q12. What is the dual-write pattern for data migration?

Dual-write writes data to both the old and new data stores simultaneously. Phase 1: all writes go to both stores; reads from old store. Phase 2: backfill historical data from old to new store. Phase 3: switch reads to new store, validate consistency. Phase 4: stop writing to old store; decommission it. Risk: window of inconsistency if one write fails. Mitigation: use idempotent writes and reconciliation jobs.

Q13. What are the risks of dual-write and how do you mitigate them?

Risks: partial failure — one store updated, other fails (inconsistency). Ordering — writes may arrive out of order in distributed systems. Performance — double write latency. Mitigations: wrap dual-write in try/catch with compensating transaction, use event-driven approach instead (write to event log, consumers update both stores asynchronously), run reconciliation job comparing both stores periodically, alert on discrepancies. Outbox pattern eliminates dual-write inconsistency.

Q14. What is ArchUnit and how does it enforce service boundaries?

ArchUnit is a Java library for enforcing architectural rules as JUnit tests. It analyzes bytecode to check dependency rules: 'no class in domain package may depend on infrastructure package', 'no class in orders service may import classes from users service domain'. Tests run in CI like any other test; violations fail the build. This prevents accidental coupling from being introduced via code reviews.

Q15. What specific ArchUnit rules would you apply to microservice modules?

1. Layered architecture: web→service→domain→infrastructure (no skip layers). 2. No cross-service domain imports: orders.domain must not import users.domain. 3. No direct SQL in web layer (only repository). 4. Event classes shared only via common events module, not domain module. 5. External service clients only in

adapters package. 6. No cyclic dependencies between packages within a service.

Q16. What is a service mesh and what problems does it solve?

A service mesh is an infrastructure layer (Istio, Linkerd) that handles inter-service communication. Sidecar proxies (Envoy) intercept all traffic. Provides: mTLS (mutual TLS for all service-to-service encryption/authentication), retries and circuit breaking at infrastructure level, traffic splitting for canary deployments, distributed tracing without code changes, rate limiting and load balancing. Reduces application-level cross-cutting concern code (no Resilience4j if mesh handles retries).

Q17. What is Istio and what are its key components?

Istio is the most widely used service mesh. Key components: Envoy sidecar proxy (data plane): injected alongside every pod, intercepts all traffic. Istiod (control plane): distributes configuration to all Envoy proxies. VirtualService: routing rules (canary traffic splitting). DestinationRule: load balancing and circuit breaker policies. PeerAuthentication: mTLS enforcement. Kiali: observability dashboard for the mesh topology.

Q18. What is the 'mega service' anti-pattern?

A mega service (or macroservice) is a microservice that has grown too large — encompasses multiple bounded contexts, has hundreds of endpoints, slow to test and deploy, owned by too many teams simultaneously. Causes: 'just add it here' approach, missing DDD boundary analysis. Fix: identify sub-domains within the service using event storming, extract sub-services using Strangler Fig pattern, each owned by one team.

Q19. What is the 'nano services' anti-pattern?

Nano services are services so small they have more infrastructure overhead than business value. Symptoms: a service with 2-3 endpoints doing trivial data passing, separate deployment pipeline but no real independence, every feature requires changes across multiple nano services. Fix: merge related nano services into a cohesive service aligned with a bounded context. Rule of thumb: if a service can't be a team's full ownership domain, it's too small.

Q20. How do you identify the right size for a microservice?

Apply the 'team size' test: a service should be owned and fully understood by a small team (2-5 people). Apply the 'deploy frequency' test: a service should be deployable independently without coordination. Apply the 'bounded context' test: align with a DDD bounded context (single domain, single team). Too large: multiple teams must coordinate changes. Too small: every feature requires multi-service changes. Start coarser (fewer services) and split when teams show clear independence.

Q21. What is an anti-corruption layer (ACL) in microservices?

An anti-corruption layer is a translation layer that prevents a downstream context's model from leaking into an upstream service. When service A consumes data from legacy service B, an ACL translates B's model to A's domain model. Prevents: direct dependency on B's DTOs in A's domain, B's schema changes breaking A's domain model. Implementation: mapper/adapter in the infrastructure layer that converts between models.

Q22. What is the 'synchronous call chain' anti-pattern?

When service A calls B, which calls C, which calls D synchronously — a call chain or 'train wreck'. Problems: response time = sum of all service latencies, availability = product of all service availabilities (0.99^4 = 96%), any service timeout/failure fails the entire chain. Solution: flatten chains with async event-driven flow, CQRS read projections to avoid cross-service reads, BFF aggregation to reduce client-initiated chains.

Q23. How does event sourcing help break anti-patterns in microservices?

Event sourcing stores all state changes as immutable events. Benefits for anti-pattern resolution: replaces shared DB reads: services subscribe to events and maintain own projections (CQRS). Eliminates synchronous call chains: services react to events instead of calling each other. Enables temporal decoupling: consumers process events at their own pace. Provides audit trail for data migration validation. Trade-off: eventual consistency, complexity of event schema evolution.

Q24. What is the expand-contract (parallel change) pattern?

Used for backward-compatible schema changes: Expand: add new field/column alongside old one. Deploy. Update code to write both fields. Migrate: update all consumers to read new field. Contract: remove old field once all consumers migrated. Example: rename 'fullName' to 'displayName': add displayName, copy values, update consumers, deprecate fullName, remove fullName. Never rename a field directly — always expand-then-contract.

Q25. How do you approach decomposing a monolith into microservices?

1. Event storming: workshop to identify domain events, commands, and bounded contexts. 2. Draw context map: identify relationships (Shared Kernel, Customer-Supplier, ACL). 3. Prioritize extraction: start with low-risk, high-value, clear-boundary domains. 4. Strangler Fig: route feature traffic to new service without touching monolith. 5. Data migration: dual-write, backfill, cutover. 6. Monitor: verify new service behavior matches monolith before retiring monolith code.

Q26. What is event storming and how does it help microservices design?

Event storming is a collaborative workshop technique where domain experts and developers use sticky notes to map domain events (orange), commands (blue), aggregates (yellow), and bounded contexts (pink frames). Reveals: natural service boundaries where event flow crosses domains, data ownership (which aggregate owns which data), process flows and potential event-driven architectures. Output: context map that guides service decomposition.

Q27. What are the risks of a big-bang microservices rewrite?

Second-system effect: tendency to over-engineer the new system. Long development time with no value delivery during rewrite. Loss of subtle business logic in the monolith (undocumented edge cases). Cannot use production traffic to validate incrementally. High rollback cost if new system fails. Team knowledge loss: developers who knew the monolith's quirks may not be on the new team. Strangler Fig avoids all of these risks by migrating incrementally.

Q28. How do you handle transactional consistency when splitting the shared database?

Move from ACID transactions to sagas (compensating transactions). Choreography saga: services publish events; each reacts and publishes next event. Orchestration saga: central orchestrator calls each step; on failure, calls compensation steps. Outbox pattern ensures atomic event publishing with state change. Accept eventual consistency: quote a consistency window (e.g., 'within 1 second, 99.9% consistent'). Design UI to handle eventual consistency (loading states, optimistic updates).

Q29. What is the outbox pattern and why is it essential for migration?

The outbox pattern writes the domain event to an 'outbox' table in the same transaction as the state change. A separate relay process reads the outbox and publishes to Kafka, then marks as published. This guarantees: event is published if and only if state change committed. Critical during migration: dual-write scenarios cannot rely on two separate transactions. Debezium CDC (change data capture) can replace the relay process by reading DB transaction log.

Q30. How does Debezium help with data migration in microservices?

Debezium captures database row changes from the transaction log (binlog/WAL) and publishes them as events to Kafka without application code changes. Migration use case: attach Debezium to legacy DB, capture all change events, new service consumes and builds its own data model. No code change in legacy system. Backfill: snapshot connector publishes full table contents. Used with Strangler Fig: new service stays synchronized with legacy during transition.

Q31. What is a context map in DDD and how does it guide service decomposition?

A context map visualizes relationships between bounded contexts: Shared Kernel: two contexts share a sub-domain model — tight coupling, evolve together. Customer-Supplier: upstream (supplier) and downstream (customer) with contractual API. Anti-Corruption Layer: downstream protects itself from upstream's model. Conformist: downstream conforms to upstream's model without translation. Published Language: well-documented shared language (e.g., event schema registry). Context maps reveal which service relationships need ACLs or event-driven decoupling.

Q32. What is domain-driven design's bounded context and how does it map to microservices?

A bounded context is a linguistic boundary where a term has a single, consistent meaning. 'Order' means different things in sales context vs fulfillment context vs billing context. Each bounded context has its own model, language, and data. Microservice = bounded context: one team owns one bounded context. Services communicate via published language (API contracts, event schemas). Violations of bounded context boundaries create the distributed monolith.

Q33. What is the anti-pattern of 'too much async'?

Excessive asynchronous communication introduces: difficult debugging (no single request trace), complex error handling (compensating transactions), hard-to-test eventual consistency, event schema versioning complexity. Balance: use sync for: user-facing reads requiring immediate response, simple CRUD operations within a bounded context. Use async for: cross-bounded context state changes, heavy processing, notifications, data propagation.

Q34. How do you introduce a service mesh incrementally?

1. Start with observability-only mode: inject Envoy sidecars but allow plain HTTP. 2. Enable distributed tracing collection (no app code changes). 3. Enable metrics collection per service. 4. Gradually enable mTLS in permissive mode (allow both mTLS and plain). 5. Switch to strict mTLS once all services have sidecars. 6. Enable traffic management (retries, circuit breakers) in mesh, remove application-level equivalents. 7. Use traffic splitting for canary deployments.

Q35. What is the 'service coordination' anti-pattern?

Service coordination anti-pattern occurs when one service orchestrates multiple others via synchronous calls in a workflow — becomes a bottleneck and single point of failure. Symptoms: 'orchestrator service' with high fan-out, all services depend on it, it must be deployed with dependent services. Solutions: choreography (events), process manager (saga orchestrator as first-class service), or BPMN workflow engine (Camunda) for complex long-running processes.

Q36. How does GraphQL federation reduce chatty service calls?

GraphQL federation allows multiple microservices to contribute to a unified GraphQL schema. A gateway (Apollo Router) composes queries across services in one request. Client sends one query; gateway fans out to relevant services in parallel; combines results. Eliminates N+1 problem: no multiple REST calls from client — one GraphQL query. Each service owns its type extensions. DataLoader pattern within each service batches DB queries to prevent N+1 within a service.

Q37. What are the organizational prerequisites for successful microservices?

Conway's Law: system architecture mirrors communication structure. Prerequisites: teams aligned to bounded contexts (not layers), each team has full-stack ownership (develop, deploy, operate), autonomous team can design, build, test, deploy without external approval, DevOps culture: team owns production, on-call rotation, SLOs, platform team providing self-service infrastructure (CI/CD, service templates). Missing these: microservices create overhead without benefits.

Q38. How do you use feature flags for zero-downtime migration?

Deploy new and old code simultaneously; flag controls execution path. Steps: 1. Deploy with flag off (old path runs). 2. Enable flag for internal users (test new path). 3. Enable for X% of users (canary). 4. Monitor error rates, latency, business metrics. 5. Ramp to 100%. 6. Remove old code path after monitoring period. 7. Remove feature flag. Flag tools: LaunchDarkly, Unleash, AWS AppConfig, or simple DB-backed toggle service.

Q39. What is 'domain event' vs 'integration event'?

Domain event: event meaningful within a bounded context — OrderItemAdded, PaymentFailed. Published within the aggregate, handled within the same bounded context. Integration event: event crossing bounded context boundaries — OrderPlaced (from orders to fulfillment). Published to message broker (Kafka). Must have stable, versioned schema. Domain events stay internal; integration events are the public API of a service. Separation prevents leaking domain model details to other services.

Q40. How do you test a Strangler Fig migration in production?

Parallel run: route all requests to both legacy and new; compare responses, return legacy result. Log discrepancies with request context for debugging. Dark launch: new service processes shadow copy of traffic (no user impact). Canary: 1% real traffic to new service; compare SLO metrics to legacy. A/B test cohorts: specific user segments see new service. Success criteria: new service matches legacy error rate, latency, and business outcome metrics.

Q41. What are the database migration pitfalls specific to microservices?

Pitfall 1: Migrating schema and application code in the same deployment. Fix: schema migration must always be backward compatible with current code. Pitfall 2: Not accounting for in-flight transactions during cutover. Fix: drain connections, wait for in-flight ops to complete. Pitfall 3: Missing indexes on new store causing performance regression. Fix: benchmark new store under production-equivalent load before cutover. Pitfall 4: Forgetting to migrate archived/historical data. Audit data completeness pre-cutover.

Q42. What is the Saga pattern and when do you use it in migration?

Sagas manage distributed transactions across services without 2PC. Use during migration when: a transaction spans the legacy and new service simultaneously. Choreography saga: each service publishes an event on success; compensates on failure event. Orchestration saga: a saga orchestrator calls each step explicitly. During Strangler Fig: the legacy system participates in sagas temporarily while migrating. After full migration: simplify saga to only involve new services.

Q43. How do you handle backward compatibility during Strangler Fig migration?

Old API must remain available throughout migration. New service implements the same API contract as legacy for the features it handles. Verify with contract tests: Pact contracts capture expected behavior, both legacy and new service must pass them. Database: old schema remains readable by legacy; new service uses new schema. Dual write ensures both schemas populated during transition. No breaking API changes during migration — save breaking changes for post-migration v2.

Q44. What is the 'CQRS+event sourcing for migration' pattern?

Legacy monolith: write state to monolith DB as before. New read service: subscribe to domain events from legacy (via Debezium CDC), build own optimized read model. Benefit: new read service decoupled from legacy schema. Can migrate writes later via Strangler Fig without disrupting reads. Eventual consistency: read model lags slightly but is self-healing via event replay. Query-intensive or reporting services benefit most from this approach.

Q45. What is 'contract-first' API design and how does it support migration?

Contract-first: define OpenAPI specification before implementing the service. In migration: write the new service's API spec, validate it doesn't break existing consumers, generate both server stubs (Spring code generator) and WireMock stubs for consumers. Consumers test against WireMock stubs from day 1 — before new service is built. This parallel development shortens migration timeline and catches misunderstandings early. API spec acts as the migration's source of truth.

Q46. How do you measure the success of a microservices migration?

Deployment frequency: can each service be deployed independently multiple times per day? Change failure rate: lower is better — microservices should enable smaller, safer changes. Time to restore service: faster with smaller, isolated services. Team autonomy: team can ship features without cross-team coordination. Operational metrics: service-level error rates, latency, resource utilization per service. Business metrics: feature development speed, reduced time-to-market.

Q47. What is the 'big ball of mud' and how do you prevent it in microservices?

Big ball of mud: no recognizable architecture, spaghetti dependencies, no clear ownership. In microservices: emerges from unplanned additions, bypassing boundaries, copying code between services, no agreed-upon patterns. Prevention: ArchUnit boundary enforcement in CI, architecture decision records (ADRs) documenting decisions, regular architecture reviews, domain-driven design from the start, tech debt tracking and remediation sprints.